Bagging Predictors

Leo Breiman, *Machine Learning* 1996

Sohan Shingade · DSC 180A · Week 3

1. The task: predict diabetes from 8 numbers

Patient	Glucose	BMI	Age	…	Diabetic?
A	148	33.6	50		yes
B	85	26.6	31		no
C	183	23.3	32		yes
D	89	28.1	21		no
E	137	43.1	33		yes
F	116	25.6	30		no

768 patients like this (Pima Indians Diabetes, Kaggle). Breiman used this same dataset.

2. One decision tree is a shaky guess

Tree trained on A–F

```
glucose > 127 ?
  yes → diabetic
  no  → not diabetic
```

Swap out two patients, retrain

```
BMI > 30 ?
  yes → age > 40 ? …
  no  → not diabetic
```

Change a little data and the whole tree changes. Breiman calls this **unstable**.

The single tree in the paper got 23% of diabetes patients wrong.

3. Noisy guesses average out, but I only have one dataset

If I had 50 **independent** datasets, I could train 50 trees and let them vote. Their individual mistakes would cancel.

one dataset → one tree → one noisy guess

I can't go collect 50 more sets of 768 patients.

So: can I fake 50 datasets from the one I have?

4. Bootstrap: resample your own data, with replacement

Draw 6 patients from {A, B, C, D, E, F} at random. Put each one back before the next draw.

Copy	Draws	Repeated	Left out
1	A A C D E E	A, E	B, F
2	B B C C D F	B, C	A, E
3	A B D E F F	F	C

Each copy is the same size as the original, but some rows repeat and some are missing.

"Bootstrap": Efron, 1979. Breiman's idea was to feed these copies to a learner.

5. Why a resample is a fair fake of a new dataset

What actually happened

The world has millions of people.
My 768 patients are a **random sample** of them.
Another study would get a *different* 768.

What bootstrap imitates

Treat my 768 as if they *were* the world.
A bootstrap copy is a **random sample** of them.
Another copy is a *different* 768.

The amount a tree **wobbles** between bootstrap copies is close to how much it would wobble between real studies. That is the whole bet.

It works because the 768 are a decent stand-in for the population. With 6 patients it would not.

6. Each copy sees about 63% of the patients

Chance a given patient is **never** drawn in 768 draws: $(1 - 1/768)^{768} \approx 1/e \approx$ **37%**.

So each bootstrap copy contains about **63%** of the distinct patients, with the rest filled by repeats.

different 63% → different tree

That is exactly what we want: 50 copies give 50 genuinely different trees.

7. Bagging = bootstrap + aggregate

```
for b in 1 … 50:
    copy_b = bootstrap sample of the data
    tree_b = train a tree on copy_b

to predict a new patient:
    ask all 50 trees
    label  →  majority vote
    number →  average
```

Nothing inside the tree learner changes. Breiman: all you add is "a loop in front … and a back end that does the aggregation."

8. Three trees vote on a new patient

New patient: glucose 130, BMI 35, age 45.

Tree	Trained on	Its first question	Says
1	copy 1	glucose > 127?	diabetic
2	copy 2	BMI > 30?	diabetic
3	copy 3	age > 48?	not diabetic

$$2 - 1 \rightarrow \text{diabetic}$$

A 2–1 vote also says something a single tree never can: **this one isn't certain.**

9. Breiman's results: better on every dataset

Dataset	One tree	50 bagged	Decrease
waveform	29.0	19.4	33%
heart (UCSD Medical Center)	10.0	5.3	47%
breast cancer	6.0	4.2	30%
ionosphere	11.2	8.6	23%
diabetes	**23.4**	**18.8**	**20%**
glass	32.0	24.9	22%
soybean	14.5	10.6	27%

Test error %, one CART tree vs. 50 bagged trees, averaged over 100 random train/test splits.

10. Bagging regression trees: average instead of vote

Same recipe, numeric target: 25 bootstrap copies, average the 25 predictions, mean squared error on the test set, 100 random splits.

Dataset	Single tree	Bagged	Decrease
Boston housing	19.1	11.7	39%
Ozone	23.1	18.0	22%
Friedman #1 (simulated)	11.4	6.2	46%
Friedman #2 (simulated)	30,800	21,700	30%
Friedman #3 (simulated)	.0403	.0249	38%

Units differ per dataset, so only the decrease column is comparable. Bagging won on all five.

§4 WHY BAGGING WORKS

11. Why it works: averaging cancels noise

Unstable learner (tree)

50 copies → 50 different trees

Averaging their answers removes the wobble

big gain

Stable learner (nearest neighbour)

50 copies → 50 nearly identical answers

Nothing to average away

no gain

Breiman's proof: the squared error of the average is never above the average squared error. The gap is the learner's variance. No variance, no gap.

> *"Bagging unstable classifiers usually improves them. Bagging stable classifiers is not a good idea."*

12. The theorem: aggregating never raises squared error

$\phi(x, L)$ = model trained on learning set L. $\phi_A(x) = E_L\, \phi(x, L)$ = the aggregated model.

$$e_A \leq e$$

Proof in one line. For any random Z, $(E\, Z)^2 \leq E\, Z^2$. With $Z = \phi(X, L)$:
$E_L\, \phi^2(X, L) \geq [E_L\, \phi(X, L)]^2 = \phi_A(X)^2$, so $E(Y - \phi)^2 \geq E(Y - \phi_A)^2$.

The gap is the variance of ϕ across learning sets. Unstable: big gap. Stable: no gap.

Classification. Voting amplifies whatever the learner does *most often* on x. Usually right → nearly always right. Usually wrong → always wrong. Bagging makes good classifiers near-optimal; it cannot rescue bad ones.

13. When bagging hurts

Section 5: linear regression, 30 variables, 60 examples. The learner picks the best m variables, then fits a line.

Variables used	Learner is	Result
few	unstable: small data changes swap which get picked	**bagged wins**
all 30	stable: nothing to pick	**bagged loses**

The error curves **cross**. Once the gain shrinks to nothing, the cost of training on resamples instead of real data takes over.

Second limit: when the single model was already near-optimal (only 3 variables mattered), bagging gave **no** improvement.

unstable *and* not already optimal

14. Concluding remarks: three leftover questions

How many copies? (Table 5.1, waveform)

Copies	Error %
0 (one tree)	29.0
10	21.8
25	19.5
50	19.4
100	19.4

> *"More than 25 bootstrap replicates is love's labor lost."*

Does it help a stable learner? (Table 5, nearest neighbour)

Dataset	Single	Bagged
waveform	26.1	26.1
heart	6.3	6.3
diabetes	16.4	16.4

Identical to the decimal, on every dataset.

Vote, or average the class probabilities? Same accuracy. But the averaged probabilities themselves are 34% more accurate.

MY IMPLEMENTATION

15. My run: same story on the diabetes data

50 random 80/20 splits, sklearn trees. Error is mean ± 95% CI.

Method	Test error %	Train (ms)	Predict (μs / patient)
one tree	30.1 ± 1.0	1.1	0.2
50 bagged trees	**23.9 ± 0.8**	45	9.6
nearest neighbour	29.5 ± 0.8	0.2	2.9
50 bagged NN	29.4 ± 0.8	18	106

- **Accuracy:** 21% less error (Breiman: 20%). Bagging won on 49 of 50 splits.
- **Confidence:** when the 50-tree vote is lopsided (>45–5) it is right **92%** of the time; when it is close (<30–20) only **57%**.
- **Speed:** ~40× slower to train and predict, but still 10 μs per patient.

TAKEAWAYS

16. Takeaways

1. **Bootstrap** fakes new datasets by resampling the one you have. It is a fair fake because your sample stands in for the population.
2. **Bagging** = bootstrap copies + vote. The learner is untouched.
3. It fixes **unstable** learners (trees: –20% to –47%). Stable ones gain nothing.
4. The **vote split** is a free confidence estimate.
5. Each copy trains with **zero communication** to the others. Breiman: "almost a dream procedure for parallel computing." That is why it maps onto a swarm of learner agents and one coordinator that votes.

Code: `slides/bagging_1996/bagging_pima_diabetes.ipynb`